SKILL WORKBOOK

24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

TEAM FSAD
K L UNIVERSITY | VADDESWARAM

SKILL WORKBOOK

24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L - FSAD

STUDENT NAME	Manan Shah
STUDENT ID	2400090137
YEAR	2nd
SEMESTER	EVEN
SECTION	
FACULTY NAME	Sridevi Sakhamuri

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Date of the Session: __/__/_____ **Time of The Session: _____to_____**

#SKILL - 4 ➔ Spring Dependency Injection – Constructor & Setter Injection

Prerequisites:

- **Basic knowledge of Java**
- **General Idea on Spring Framework**
- **Basic Idea on DI & IoC**

A training institute wants to automate the management of student information in a Spring application. The system should inject details such as **studentId**, **name**, **course**, and **academic year** into objects without manually assigning values. This Skill demonstrates **Constructor Injection** and **Setter Injection** using both **XML** and **Annotation** configurations.

Tasks:

1. Create a Java class named **Student** with fields: studentId, name, course, year.
2. Create a **Constructor** that accepts all fields and assigns values.
3. Create **Setter methods** for at least two fields (example: course, year).
4. Configure the Student bean using:
   a. **XML Configuration** - The student should take:
      - a **POJO class**
      - an **XML configuration file**
      - a **main class** to load the container
   b. **Annotation Configuration** - The student should take:
      - a **POJO class**
      - a **Java class with necessary annotations**
      - a **main class** to load the container
5. Create a Spring configuration file (XML or Java-based).
6. Write a main class to load the Spring IoC container.
7. Retrieve the Student bean and print the injected values.
8. **Push the Spring (core) project to GitHub.**

PART A – XML Configuration

Student.java (POJO Class)

```
package com.institute.model;

public class Student {

    private int studentId;
    private String name;
    private String course;
    private int year;

    // Constructor Injection
    public Student(int studentId, String name, String course, int year) {
        this.studentId = studentId;
        this.name = name;
        this.course = course;
```

```
        this.year = year;
    }

    // Setter Injection
    public void setCourse(String course) {
        this.course = course;
    }

    public void setYear(int year) {
        this.year = year;
    }

    public void display() {
        System.out.println("Student ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Year: " + year);
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="student" class="com.institute.model.Student">
        <!-- Constructor Injection -->
        <constructor-arg value="101"/>
        <constructor-arg value="Ashu Kumar"/>
        <constructor-arg value="CSE"/>
        <constructor-arg value="1"/>

        <!-- Setter Injection -->
        <property name="course" value="Computer Science"/>
        <property name="year" value="2"/>
    </bean>
```

```
</beans>
```

MainApp.java

```
package com.institute.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.institute.model.Student;

public class MainApp {
   public static void main(String[] args) {

      ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

      Student s = (Student) context.getBean("student");
      s.display();
   }
}
```

Part B - Annotation Configuration

Student.java (With Annotations)

```
package com.institute.model;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class Student {

   private int studentId;
   private String name;
```

```
    private String course;
    private int year;

    // Constructor Injection
    public Student(@Value("102") int studentId,
                   @Value("Rahul") String name,
                   @Value("IT") String course,
                   @Value("1") int year) {
        this.studentId = studentId;
        this.name = name;
        this.course = course;
        this.year = year;
    }

    // Setter Injection
    @Value("AI & ML")
    public void setCourse(String course) {
        this.course = course;
    }

    @Value("3")
    public void setYear(int year) {
        this.year = year;
    }

    public void display() {
        System.out.println("Student ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Year: " + year);
    }
}
```

SpringConfig.java

```
package com.institute.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
@ComponentScan("com.institute")
public class SpringConfig {
}
```

MainApp.java (Annotation)

```
package com.institute.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.institute.config.SpringConfig;
import com.institute.model.Student;

public class MainApp {
   public static void main(String[] args) {

      ApplicationContext context =
            new AnnotationConfigApplicationContext(SpringConfig.class);

      Student s = context.getBean(Student.class);
      s.display();
   }
}
```

VIVA QUESTIONS:

1. What is Dependency Injection (DI) in Spring?

   Dependency Injection is a design pattern used in Spring where the framework provides the required objects (dependencies) to a class instead of the class creating them itself. This helps in reducing tight coupling between classes and makes the application easier to test and maintain.

2. What is the difference between Constructor DI and Setter DI?

   In Constructor DI, dependencies are provided through the class constructor at the time of object creation, making them mandatory. In Setter DI, dependencies are provided using setter methods after the object is created, making them optional. Constructor DI ensures that required dependencies are always available, while Setter DI offers more flexibility.

3. What is the purpose of the IoC container?

   The IoC (Inversion of Control) container in Spring is responsible for creating objects, managing their lifecycle, and injecting dependencies into them. It controls how objects are created and connected, which removes the responsibility from the developer and simplifies application development.

4. Why is DI preferred over manual object creation?

   DI is preferred because it reduces coupling between classes, improves code reusability, and makes unit testing easier. When objects are created manually using new, classes become tightly connected, but DI allows Spring to manage dependencies, making the system more flexible.

5. Can Constructor Injection and Setter Injection be used together?

   Yes, both Constructor Injection and Setter Injection can be used together in the same class. Usually, Constructor Injection is used for required dependencies, while Setter Injection is used for optional ones.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Date of the Session: __/__/_____ **Time of The Session: _____to_____**

#SKILL - 5 ➔ Spring Autowiring Demo using @Autowired

Prerequisites:

- **General Idea on Spring Framework**
- **Modules of Spring Framework**
- **Understanding of Autowiring concepts**

A college application maintains **student** information along with their **certification details**. The **Student** object contains a **Certification** object, and instead of creating this Certification object manually, Spring should automatically provide it using the **@Autowired** annotation. When the required classes are marked with **@Component**, Spring creates their objects and manages them inside the **IoC container**. **Using @Autowired**, the Certification object is supplied to the **Student** object by Spring without using the **new keyword**, allowing both objects to be linked through **annotation-based configuration.**

Tasks:

1. Create a **Certification** class with fields such as id, name, and dateOfCompletion.
2. Create a **Student** class with fields such as id, name, gender, and a Certification object.
3. Annotate both classes with **@Component** to enable Spring detection.
4. Use **@Autowired** on a field, constructor, or setter inside the Student class to inject the Certification object.
5. Configure component scanning using either:
   a. An XML configuration file
   b. A Java configuration class with @Configuration and @ComponentScan
6. Load the Spring IoC container using ApplicationContext.
7. Retrieve the Student bean and print all details, including the injected Certification object.
8. **Push the Spring project to GitHub.**

Certification.java

```
package com.college.model;

import org.springframework.stereotype.Component;

@Component
public class Certification {

   private int id = 501;
   private String name = "Java Full Stack";
   private String dateOfCompletion = "10-Feb-2026";

   public String toString() {
      return "Certification ID: " + id +
            ", Name: " + name +
            ", Completed On: " + dateOfCompletion;
   }
}
```

Student.java

```
package com.college.model;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Student {

   private int id = 101;
   private String name = "Ashu Kumar";
   private String gender = "Male";
   @Autowired   // Spring injects Certification object here
```

```
    private Certification certification;

    public void display() {
        System.out.println("Student ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Gender: " + gender);
        System.out.println(certification); // calls toString()
    }
}
```

SpringConfig.java

```
package com.college.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.college")
public class SpringConfig {
}
```

MainApp.java

```
package com.college.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.college.config.SpringConfig;
import com.college.model.Student;

public class MainApp {
    public static void main(String[] args) {
```

```
        ApplicationContext context =
                new AnnotationConfigApplicationContext(SpringConfig.class);

        Student s = context.getBean(Student.class);
        s.display();
    }
}
```

VIVA QUESTIONS:

1. What is autowiring in Spring?

   Autowiring is a feature in Spring that automatically connects dependent objects (beans) without the developer manually specifying them in configuration files. Spring looks at the dependency type and injects the required bean automatically, which reduces configuration work.

2. What is the purpose of the @Autowired annotation?

   The @Autowired annotation is used to tell Spring to automatically inject a required dependency into a class. It can be used on constructors, setter methods, or fields to let Spring know where the dependency should be provided.

3. How does Spring choose which bean to inject?

   Spring mainly chooses a bean based on the **type** of the dependency. If only one bean of that type exists in the container, it injects it automatically. It can also use the bean name or @Qualifier annotation to choose a specific bean when needed.

4. What is component scanning in Spring?

   Component scanning is a process where Spring automatically detects classes annotated with stereotypes like @Component, @Service, @Repository, and @Controller in a specified package. It then registers them as beans in the Spring container.

5. What happens if more than one bean matches the dependency type?

   If more than one bean of the same type exists, Spring gets confused and throws a **NoUniqueBeanDefinitionException**. To solve this, we use the @Qualifier annotation or mark one bean as @Primary to tell Spring which one to inject.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Date of the Session: __/__/_____ **Time of The Session: _____to_____**

#SKILL - 6 ➔ Spring MVC Web Request Handling Demo

Prerequisites:

- **General Idea on Spring Web MVC and Form Handling**
- **Basic understanding of Spring Boot**
- **Understanding of HTTP methods (GET/POST)**

An online library system wants to allow users to explore books, view specific book details, submit feedback, and interact with the system through various types of requests. To support these features, the backend team needs to implement multiple **Spring MVC endpoints** demonstrating **request mappings**, **path variables**, **request parameters**, and **JSON data handling**. Your task is to create a **Spring Boot MVC controller** that exposes different demo operations using **@RestController**, **@GetMapping**, **@PostMapping**, and **@PathVariable**.

Tasks:

1. Create a controller class named **LibraryController**.
2. Create a method mapped to **/welcome** that returns a welcome message.
3. Create a method mapped to **/count** that returns an integer representing total books.
4. Create a method mapped to **/price** that returns a sample book price as **double**.
5. Create a method mapped to **/books** using **@GetMapping** that returns a list of book titles.
6. Create a method mapped to **/books/{id}** using **@GetMapping** and return book details using **@PathVariable**.
7. Create a method mapped to **/search** using **@GetMapping** that accepts a **request parameter** (title) and returns a confirmation message.
8. Create a method mapped to **/author/{name}** using **@GetMapping** that returns a formatted message with the **author's name**.
9. Create a method mapped to **/addbook** using **@PostMapping** that accepts a **Book object** from the request body and adds it to an **in-memory list**.
10. Create a method mapped to **/viewbooks** using **@GetMapping** that returns all added **Book objects**.
11. **Push the Spring Boot project to GitHub.**

LibraryMvcAppApplication.java

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryMvcAppApplication {
   public static void main(String[] args) {
      SpringApplication.run(LibraryMvcAppApplication.class, args);
   }
}
```

Book.java

```
package com.library.model;

public class Book {

   private int id;
   private String title;
   private String author;
   private double price;

   public Book() {}

   public Book(int id, String title, String author, double price) {
      this.id = id;
      this.title = title;
      this.author = author;
      this.price = price;
```

```
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

LibraryController.java

```
package com.library.controller;

import org.springframework.web.bind.annotation.*;
import com.library.model.Book;
import java.util.*;

@RestController
public class LibraryController {

    private List<Book> bookList = new ArrayList<>();

    // 2. Welcome message
    @GetMapping("/welcome")
    public String welcome() {
        return "Welcome to Online Library System!";
    }

    // 3. Total book count
    @GetMapping("/count")
    public int bookCount() {
        return 150;
    }
```

```
// 4. Sample price
@GetMapping("/price")
public double bookPrice() {
    return 499.99;
}

// 5. List of titles
@GetMapping("/books")
public List<String> getBooks() {
    return Arrays.asList("Java Basics", "Spring Boot Guide", "Data Structures");
}

// 6. Book details by ID
@GetMapping("/books/{id}")
public String bookById(@PathVariable int id) {
    return "Details of Book ID: " + id;
}

// 7. Search by title
@GetMapping("/search")
public String searchBook(@RequestParam String title) {
    return "Searching for book titled: " + title;
}

// 8. Author name
@GetMapping("/author/{name}")
public String author(@PathVariable String name) {
    return "Books written by author: " + name;
}

// 9. Add book (POST JSON)
@PostMapping("/addbook")
public String addBook(@RequestBody Book book) {
    bookList.add(book);
    return "Book added successfully!";
}

// 10. View all added books
@GetMapping("/viewbooks")
public List<Book> viewBooks() {
    return bookList;
}
```

```
}
```

VIVA QUESTIONS:

1. What is the role of @RestController in Spring MVC?

   @RestController is used to create RESTful web services in Spring MVC. It combines @Controller and @ResponseBody, which means the methods in the class return data (like JSON or XML) directly instead of returning a web page. It is mainly used for building APIs.

2. What is the difference between @GetMapping and @PostMapping?

   @GetMapping is used to handle HTTP GET requests, which are mainly used to retrieve data from the server. @PostMapping is used to handle HTTP POST requests, which are used to send data to the server, such as submitting forms or saving information in a database.

3. How does @PathVariable help in handling dynamic URLs?

   @PathVariable is used to extract values from the URL path and pass them to a method. It helps in handling dynamic URLs, such as /student/101, where 101 can be captured as a method parameter and used inside the program.

4. When do we use @RequestParam?

   @RequestParam is used to get values from query parameters in the URL. For example, in /search?name=Ashu, the value of name can be accessed using @RequestParam. It is commonly used for filtering, searching, or passing small pieces of data.

5. Can one controller contain multiple request-mapped methods?

   Yes, a single controller can contain multiple methods with different request mappings. Each method can handle a different URL or HTTP request type, which helps organize related functionalities in one controller class.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: